

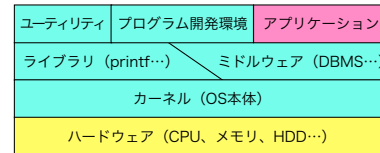
オペレーティングシステムの機能を使ってみよう 第1章 システムプログラミング

オペレーティングシステムの機能を使ってみよう

1 / 6

システムプログラムとは

- カーネル (OS の本体)
- ライブラリ (プログラムが使用するサブルーチン, DLL ...)
- ミドルウェア (DBMS, Web サーバ ...)
- ユーティリティ (ファイル操作, 時計, シェル, システム管理 ...)
- プログラム開発環境 (エディタ, コンパイラ, アセンブラ, リンカ, インタプリタ ...)



オペレーティングシステムの機能を使ってみよう

2 / 6

システムプログラミングとは

- システムプログラムを作成すること。
- 本講義ではユーティリティのプログラミングを行う。

なぜシステムプログラミング？

「システムプログラミングを通してオペレーティングシステムの体感的な理解」をする。

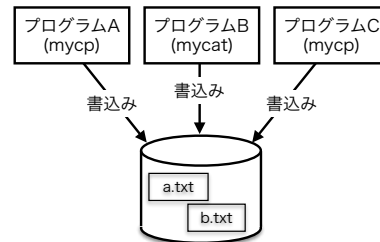
オペレーティングシステムを体感的に理解するために、オペレーティングシステムの機能を直接使用する簡単な CLI 版のユーティリティプログラムの作成 (プログラミング) を行う。

オペレーティングシステムの機能を使ってみよう

3 / 6

複数のプログラムが正しく実行されない

- コンピュータの中では複数のプログラムが同時に作動している。
- 各プログラムが勝手に資源にアクセスすると具合が悪い。
- 例えばディスクのどの領域をどのファイルが使用するか？
各プログラムが勝手に決めると不具合が起こる。

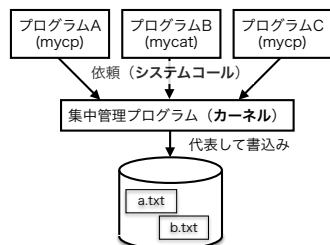


オペレーティングシステムの機能を使ってみよう

4 / 6

複数のプログラムが正しく実行される

- OS の本体 (カーネル) が代表して資源を管理する。
- 一般のプログラムはカーネルに処理を依頼し目的を達成する。
- 例えばファイルを作成してディスクのどの領域を使用するか？
カーネルが責任を持って一貫した管理を行う。(集中管理)
- 他のプログラムはシステムコールを行いカーネルに処理を依頼。



オペレーティングシステムの機能を使ってみよう

5 / 6

システムコールの使用

- C 言語からシステムコールを利用することができる。
- UNIX (macOS) の C 言語ではシステムコールと同じ名前の関数を呼び出すとシステムコールの発行になる。
- macOS 上で C 言語を用いてシステムコールを直接使用するユーティリティプログラムを作成する (システムプログラミングを行う)。
- OS の機能をシステムコールを通して実感する。
- OS が提供すべき機能を理解する。

```
// ディレクトリを作るユーティリティプログラム (mymkdir) の例
int main(int argc, char *argv[]) {
    if (argc != 2) {
        // エラー処理
    }
    mkdir(argv[1]); // ディレクトリを作るシステムコール
    return 0;
}
```

オペレーティングシステムの機能を使ってみよう

6 / 6

オペレーティングシステムの機能を使ってみよう 第2章 ファイル入出力システムコール

ファイル入出力システムコール

- ファイルの読み書きを行うシステムコールを勉強する。
- システムコールを直接使用したプログラムを作成してみる。
- プログラムの作成にはC言語を用いる。
- システムコールを直接使用する入出力を低水準入出力と呼ぶ、これまで使用してきたものは高水準入出力と呼ぶ。

高水準入出力と低水準入出力

- C言語の入門で勉強した入出力関数は高水準入出力関数。
- システムコールを直接使用する入出力は低水準入出力。
- 高水準入出力関数は内部でシステムコールを利用。
- 高性能・豊富な高水準と、シンプルな低水準。

高水準入出力関数	対応するシステムコール
fopen()	open システムコール
printf()	write システムコール
putchar()	write システムコール
fputs()	write システムコール
fputc()	write システムコール
...	...
scanf()	read システムコール
getchar()	read システムコール
fgets()	read システムコール
fgetc()	read システムコール
...	...
fclose()	close システムコール

open システムコール (書式1)

- ファイルを開くシステムコール
- fopen() 関数が使用している

書式 (オープンする場合)

```
#include <fcntl.h>
int open(const char *path, int oflag);
```

解説 (書式1, 2 共通)

- fcntl.h をインクルードする必要がある。
- open システムコールは正常時にはファイルディスクリプタ (3以上の番号) を返す¹。
- エラーが発生した時は-1を返す。
- エラー原因は perror() 関数で表示できる。

¹stdin が0, stdout が1, stderr が2なので3以降になる。

- 引数**
- path はオープンまたは作成するファイルのパス (名前)
 - oflag はオープンする方法を表の記号定数を「|」で接続して書く。 (「|」は、C言語のビット毎の論理和演算子)

以下の一つ	と	以下のいくつか
O_RDONLY (読み出し用)		O_APPEND (追記)
O_WRONLY (書き込み用)	+	O_CREAT (作成)
O_RDWR (読み書き両用)		O_TRUNC (切詰め)
		...

使用例

```
#include <fcntl.h>
...
int fdr, fdw, fda;
fdr=open("r.txt", O_RDONLY); // 読み出し用にオープン
fdw=open("w.txt", O_WRONLY); // 書き込み用にオープン
fda=open("a.txt", O_WRONLY|O_APPEND); // 追記用にオープン

if (fdw<0) { // エラーチェック
    perror("w.txt"); // 原因の表示
    exit(1); // エラー終了
}
```

open システムコール (書式2)

ファイルが存在しない時はファイルを自動的に作った上で開く。

書式 (ファイル作成もする場合)

oflag に O_CREAT を含む場合は、該当ファイルが存在しないなら新規作成してからオープンする。新規作成するファイルの保護モードを mode で指定する。

```
#include <fcntl.h>
int open(const char *path, int oflag, mode_t mode);
```

- mode_t は、16bit の整数型 (16bit int) である。
- mode は、作成されるファイルの保護モードである。
- mode は、8進数で記述することが多い。

```
0: --- 2: -w- 4: r-- 6: rw-
1: --x 3: -wx 5: r-x 7: rwx
```

使用例

```
fd=open("a.txt", O_WRONLY|O_CREAT, 0644); // モードは rw-r--r--
```

ファイルの保護モード

open システムコールの第3引数 (mode) は次のような 12bit の値である。

11	10	9	8	7	6	5	4	3	2	1	0
s	s	t	r	w	x	r	w	x	r	w	x
省略			ユーザ			グループ			その他		

- 最初の 3bit の意味は難しいのでここでは説明を省略する。
- 他のビットは `rwX` のどれかである。 `rwX` の意味は次の通り。

`r` : **read** 可 (読み出し可能)
`w` : **write** 可 (書き込み可能)
`x` : **execute** 可 (実行可能)

例えば, 第8ビットが1だった。⇒
 ユーザ (ファイルの所有者) が read (読み出し) 可能の意味。

ファイルのモードやユーザ (所有者), グループは次のようにして確認できる。

```
% ls -l a.txt
-rw-r--r-- 1 sigemura staff 0 Apr 11 05:53 a.txt
%
```

実行結果から `a.txt` ファイルについて以下のことが分かる。

- モードの下位 9 ビットが 110100100 である。
- 所有者は sigemura である。
- グループは staff である。

以上を総合すると `a.txt` ファイルについて以下のことが分かる。

- sigemura が読み書きができる。
- staff グループに属するユーザは読むことだけできる。
- その他のユーザも読むことだけできる。

read システムコール (1)

- 読み出し用にオープン済みのファイルからデータを読む。
- ファイルの先頭から順に読み出す。
 (シーケンシャルアクセス (順アクセス))

書式 (詳しくは `man 2 read` で調べる.)

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t nbyte);
```

解説

- `unistd.h` をインクルードする必要がある。
- `ssize_t` は 64bit 整数型。
- `size_t` は 符号なし 64bit 整数型。
- 正常時には読んだデータのバイト数 (正の値) を返す。
- EOF では 0 を返す。
- エラーが発生した時は -1 を返す。
- エラーの原因は `perror()` 関数で表示できる。

read システムコール (2)

引数

- `fd` はオープン済みのファイルディスクリプタ
- `buf` はデータを読み出すバッファ領域を指すポインタ
- `nbyte` はバッファ領域の大きさ (バイト単位)

使用例 1

- `fd` は `open` システムコールでオープン済みと仮定
- `buf` はバッファ用の `char` 型の大きさ 100 の配列
- `char` 型は 1 バイトなので, 配列全体で 100 バイト
- 3 回の `read` によりファイルの先頭から順に 100 バイトずつ読む

```
char buf[100];
n = read(fd, buf, 100); // 1回目
n = read(fd, buf, 100); // 2回目
n = read(fd, buf, 100); // 3回目
```

read システムコール (3)

使用例 2

- ループでファイルの先頭から順にデータを読み出す例
- `n` の値が 0 以下になったら EOF かエラー
- EOF かエラーになったらループを終了

```
while ((n=read(fd, buf, 100)) > 0) { // 読む
    ... 読んだ n バイトのデータを処理する ...
}
```

write システムコール

- 書き込み用にオープン済みのファイルヘデータを書き込む。
- ファイルの先頭から順にデータを書き込む。
 (シーケンシャルアクセス)
- ファイルの最後に達するまでは元々あったデータを上書きする。
- ファイルの最後に書き込むとファイル長が延長される。

書式 (詳しくは `man 2 write` で調べる.)

```
#include <unistd.h>
ssize_t write(int fd, void *buf, size_t nbyte);
```

解説

- ファイルに実際に書き込んだデータのバイト数を返す。
- 返された値が `nbyte` と一致しない場合はエラー?

使用例 ファイルに `abc` の 3 バイトを書き込む。

```
char *a = "abc";
n = write(fd, a, 3); // nが3以外ならエラーが疑われる
```

lseek システムコール (1)

- オープン済みファイルの読み書き位置を移動する.
- lseek システムコールと組み合わせることで, read, write システムコールを用いたファイルのランダムアクセス (直接アクセス) が可能になる.

書式 (詳しくは man 2 lseek で調べる.)

```
#include <unistd.h>
off_t lseek(int fd, off_t offset, int whence);
```

解説

- fd はオープン済みのファイルディスクリプタ
- off_t 型は 64bit int 型
- ファイルの現在の読み書き位置を offset に移動
- offset の意味は whence によって変化する.
- 正常時は新しい読み書き位置が返される.
- エラーが発生した時は -1 を返す.
- エラーの原因は perror() 関数で表示できる.

オペレーティングシステムの機能を使ってみよう

13 / 15

lseek システムコール (2)

whence の意味

whence	意味
SEEK_SET	offset はファイルの先頭からのバイト数
SEEK_CUR	offset は現在の読み書き位置からのバイト数
SEEK_END	offset はファイルの最後からのバイト数

使用例 fd はオープン済みのファイルディスクリプタとする.

```
lseek(fd, 200, SEEK_SET); // 先頭から200バイトに移動する.
lseek(fd, -100, SEEK_CUR); // 現在地から100バイト先頭方向に移動する.
lseek(fd, -10, SEEK_END); // 最後から10バイト先頭方向に移動する.
```

オペレーティングシステムの機能を使ってみよう

14 / 15

close システムコール

ファイルを閉じる.

書式 (詳しくは man 2 close で調べる.)

```
#include <unistd.h>
int close(int fd);
```

解説

- オープン済みのファイルを閉じる.
- 引数はオープン済みのファイルディスクリプタ
- 多数のファイルを開くプログラムでは不要になったものをクローズしないと, 同時に開くことができるファイル数の上限を超えることがある.

使用例 fd はオープン済みのファイルディスクリプタとする.

```
close(fd);
```

オペレーティングシステムの機能を使ってみよう

15 / 15

オペレーティングシステムの機能を使ってみよう 第3章 高水準入出力と低水準入出力

オペレーティングシステムの機能を使ってみよう

1/9

高水準入出力と低水準入出力

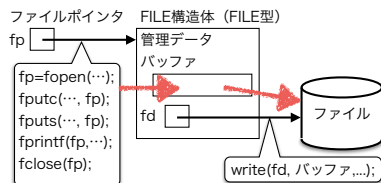
ファイルを読み書きするための機能
(API: Application Program Interface)

- 高水準入出力 (高水準 I/O)
豊富, かつ, 高機能な API
(`fprintf()`, `fscanf()`, `fputc()`, `fgetc()`, ...)
- 低水準入出力 (低水準 I/O)
システムコールのこと
少なく, かつ, シンプルな API
(`open()`, `read()`, `write()`, `lseek()`, `close()`)

オペレーティングシステムの機能を使ってみよう

2/9

高水準 I/O のデータ構造 (書き込み)

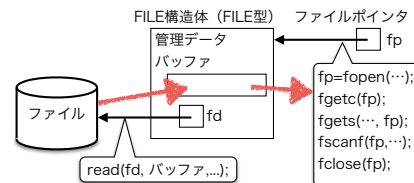


- ファイルポインタ (fp)
- FILE 構造体
- バッファリング
- write システムコール

オペレーティングシステムの機能を使ってみよう

3/9

高水準 I/O のデータ構造 (読み出し)



- ファイルポインタ (fp)
- FILE 構造体
- read システムコール
- バッファリング

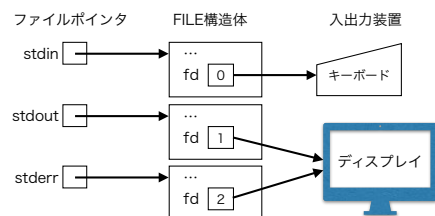
オペレーティングシステムの機能を使ってみよう

4/9

標準入出力 (標準入出力カストリーム)

名称	fd	fp	通常の接続先
標準入力ストリーム	0	stdin	キーボード
標準出力ストリーム	1	stdout	ディスプレイ
標準エラー出力ストリーム	2	stderr	ディスプレイ

fd: ファイルディスクリプタ
fp: ファイルポインタ



オペレーティングシステムの機能を使ってみよう

5/9

ユニファイド I/O

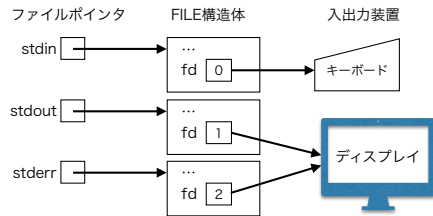
標準ストリーム	同じ意味の呼出し	役割
<code>scanf(...)</code>	<code>fscanf(stdin, ...)</code>	書式付きの入力
<code>getchar()</code>	<code>fgetc(stdin)</code>	1 文字入力
<code>-</code>	<code>fgets(stdin, ...)</code>	1 行入力
<code>printf(...)</code>	<code>fprintf(stdout, ...)</code>	書式付きの出力
<code>putchar(c)</code>	<code>fputc(c, stdout)</code>	1 文字出力
<code>puts(buf)</code>	<code>fputs(buf, stdout)</code>	1 行出力

- `printf(...)` と `fprintf(stdout, ...)` は同じ
- fp の代わりに `stdin`, `stdout` 等が使用できる.
- キーボードやディスプレイ (入出力装置) とファイルを同じ要領で操作できる.
- 入出力装置をファイルに統合 = (ユニファイド I/O)

オペレーティングシステムの機能を使ってみよう

6/9

標準入力ストリーム

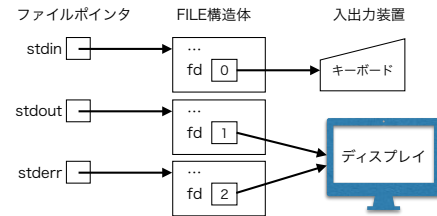


- ファイルポインタは `stdin`
- ファイルディスクリプタは 0 番
- ファイルディスクリプタ 0 は通常キーボードに接続
- ファイルポインタと FILE 構造体はプログラム起動時に初期化
- シェルはファイルディスクリプタ 0 をリダイレクト可能

オペレーティングシステムの機能を使ってみよう

7/9

標準出力ストリーム

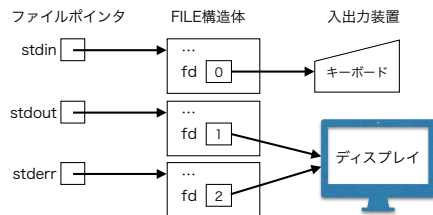


- ファイルポインタは `stdout`
- ファイルディスクリプタは 1 番
- ファイルディスクリプタ 1 は通常ディスプレイに接続
- ファイルポインタと FILE 構造体はプログラム起動時に初期化
- シェルはファイルディスクリプタ 1 をリダイレクト可能

オペレーティングシステムの機能を使ってみよう

8/9

標準エラー出力ストリーム



- エラーメッセージ出力用のストリーム
- ファイルポインタは `stderr`
- ファイルディスクリプタは 2 番
- ファイルディスクリプタ 2 は通常ディスプレイに接続
- ファイルポインタと FILE 構造体はプログラム起動時に初期化
- シェルはファイルディスクリプタ 2 をリダイレクト可能

オペレーティングシステムの機能を使ってみよう

9/9

オペレーティングシステムの機能を使ってみよう 第4章 ファイルシステム

オペレーティングシステムの機能を使ってみよう

1 / 24

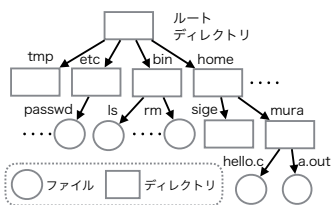
ファイルシステム

- ファイル
二次記憶装置（ストレージ）に格納された不揮発性のデータ記憶
（二次記憶装置：HDD, USB メモリ, CD-ROM, SSD, ...）
- ファイルシステム
二次記憶装置に多数のファイルを記憶・管理する仕組み
記憶・管理されているファイルの集合
- UNIX ファイルシステム
Windows や macOS のファイルシステムも基本は同じ

オペレーティングシステムの機能を使ってみよう

2 / 24

ファイル木

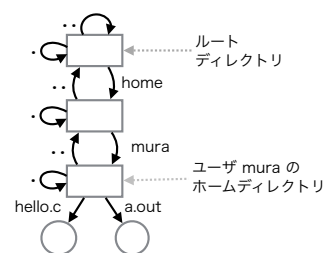


- ルートディレクトリを根にした有向の木構造
- 節点（ノード）はディレクトリ（フォルダ）
- 葉（リーフ）はファイル
- 有向枝（エッジ）はリンク
- ファイルとリンクは独立している
- ディレクトリもファイルの一種

オペレーティングシステムの機能を使ってみよう

3 / 24

特別なディレクトリ



- ルートディレクトリ（ファイル木の根になるディレクトリ）
- 親ディレクトリ（ルートに近いディレクトリ, 「..」で表す）
- カレントディレクトリ（現在位置, 「.」で表す）
- ホームディレクトリ（ログイン時のカレントディレクトリ）

オペレーティングシステムの機能を使ってみよう

4 / 24

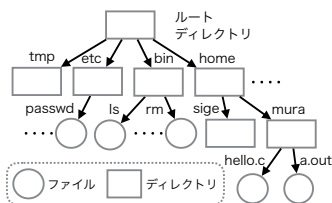
パス (Path)

- パスは径の意味（ファイルへの道）
- ファイルをパスにより特定できる。
- ファイル木のリンクに付いた名前を「/」で区切って書く。
- パスには以下の二種類がある。
 - 1 絶対パス
ルートディレクトリを起点にしたパス
「/」で書き始める。
例：/home/mura/hello.c
 - 2 相対パス
カレントディレクトリを起点にしたパス「/」以外で書き始める。
例：hello.c

オペレーティングシステムの機能を使ってみよう

5 / 24

絶対パス



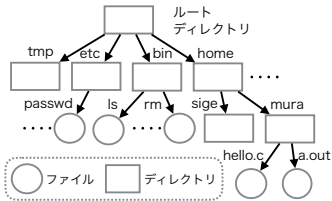
ルートディレクトリを起点にしたパス

- /etc/passwd
- /bin/ls
- /home/mura：ディレクトリへのパス
- /home/mura/hello.c
- /home/sig/../../mura/./hello.c

オペレーティングシステムの機能を使ってみよう

6 / 24

相対パス



カレントディレクトリを起点にしたパス
(カレントディレクトリが/home のとき)

- mura/hello.c
- sig: ディレクトリへのパス
- ../bin/ls
- sig/../../mura/hello.c

オペレーティングシステムの機能を使ってみよ

7 / 24

カレントディレクトリ

プロセス毎にカレント（現在の）ディレクトリがある。

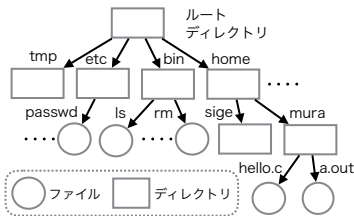
- カレントディレクトリの変更は他のプロセスに影響はない。
- 他のターミナル（ターミナルもプロセス）に影響はない。
- 次回のログインにも引継がれない。
- 以下のコマンドで変更と確認ができる。
 - 変更 (cd コマンド)
 - % cd パス
 - 確認 (pwd コマンド)
 - % pwd

オペレーティングシステムの機能を使ってみよ

8 / 24

cd, pwd コマンドの実行例

```
% pwd
/home/mura
% ls
a.out  hello.c
% ls .
a.out  hello.c
% cp hello.c h.c
% cp /home/mura/hello.c i.c
% ls
a.out  h.c
i.c    hello.c
% cd ..
% pwd
/home
% cd ../bin
% pwd
/bin
% cd ~
% pwd
/home/mura
```



オペレーティングシステムの機能を使ってみよ

9 / 24

演習 4-1

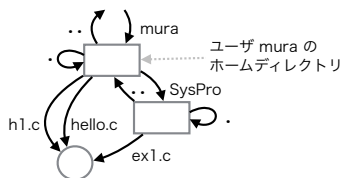
ファイル木を理解する

- Linux の人: 0410_演習1_...(Linux 版).pdf をやってみる。
- Mac の人: 0420_演習1_...(Mac 版).pdf をやってみる。

オペレーティングシステムの機能を使ってみよ

10 / 24

リンク（ハードリンク）

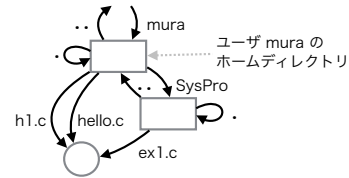


- ファイルに別名を付ける。
- ハードリンクとシンボリックリンクの二種類がある。
- ハードリンク
 - 従来のリンクと同じもの
 - 一つのファイル本体に複数のリンクが可能
 - 元々あったリンク、後で追加したリンクに区別はない

オペレーティングシステムの機能を使ってみよ

11 / 24

リンク（ハードリンクの作成）



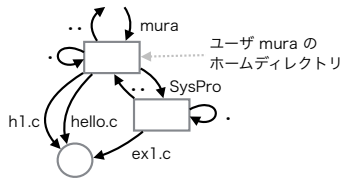
ln コマンドを用いる。

```
% pwd
/home/mura
% ln hello.c h1.c      # カレントディレクトリはここ
% mkdir SysPro         # hello.c にリンク h1.c を追加
% ln hello.c SysPro/ex1.c # SysPro ディレクトリを作る
% cat h1.c             # リンク ex1.c を追加
% cat SysPro/ex1.c     # hello.c の内容が表示される
% cat SysPro/ex1.c     # hello.c の内容が表示される
```

オペレーティングシステムの機能を使ってみよ

12 / 24

リンク (ハードリンクの削除)



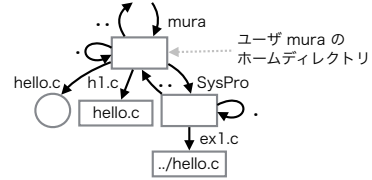
rm コマンドを用いる,

```
% pwd
/home/mura          # カレントディレクトリはここ
% rm h1.c           # リンク h1.c を削除
% rm SysPro/ex1.c   # リンク ex1.c を削除
% rmdir SysPro      # SysPro ディレクトリを削除
```

オペレーティングシステムの機能を使ってみよ

13 / 24

リンク (シンボリックリンク)



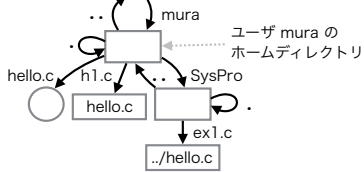
● シンボリックリンク

- パスを格納した特殊なファイル
- ソフトリンクとも呼ぶ
- パス名の解釈時に字面で評価される
- 字面での評価なので制約が少ない (別ディスク, リンク切)
- ファイルが置換わると新しいファイルを参照する

オペレーティングシステムの機能を使ってみよ

14 / 24

リンク (シンボリックリンクの作成)



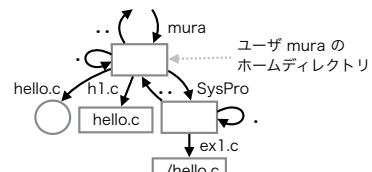
ln -s コマンドを用いる,

```
% pwd
/home/mura          # カレントディレクトリはここ
% ln -s hello.c h1.c # リンク h1.c を作成
% mkdir SysPro       # SysPro ディレクトリを作る
% ln -s ../hello.c SysPro/ex1.c # ex1.c を作成
% cat h1.c           # hello.c の内容が表示される
% cat SysPro/ex1.c   # hello.c の内容が表示される
```

オペレーティングシステムの機能を使ってみよ

15 / 24

リンク (シンボリックリンクの削除)



rm コマンドを用いる,

```
% pwd
/home/mura          # カレントディレクトリはここ
% rm h1.c           # リンク h1.c を削除
% rm SysPro/ex1.c   # リンク ex1.c を削除
% rmdir SysPro      # SysPro ディレクトリを削除
```

オペレーティングシステムの機能を使ってみよ

16 / 24

演習 4-2

リンクに関する演習

- 0440_演習2_リンクと...._演習.pdf の「1. リンクを作る」をやってみる.

オペレーティングシステムの機能を使ってみよ

17 / 24

ファイルの属性

主な属性

種類 普通ファイル, ディレクトリ, シンボリックリンク等

保護モード open システムコールで紹介した `rw-rw-rwx`.

リンク数 ファイルを指しているハードリンクの数, リンク数が0になるとファイル本体が削除される. (例えば, ハードリンク例の `hello.c` ファイルの場合は3になる)

所有者 所有者のユーザ番号.

グループ 属するグループのグループ番号.

ファイルサイズ ファイルの大きさ (バイト単位).

最終参照日時 最後にアクセスした時刻.

最終変更日時 内容を最後に変更した時刻.

最終属性変更時刻 属性を最後に変更した時刻.

オペレーティングシステムの機能を使ってみよ

18 / 24

属性の表示方法

ls -l コマンドを用いる。

```
% ls -l a.txt
-rw-r--r-- 1 mura staff 10 May 1 18:18 a.txt
```

ファイルの種類 一文字目の「-」はファイルが普通のファイルであることを表している。一文字目が「d」はディレクトリであること、「l」はシンボリックリンクであることを表す。

ファイルの保護モード open システムコールで紹介したものの (rwxrwxrwx)。

リンク数 1 はリンク数が1であることを表している。

所有者 mura はファイルの所有者がユーザ mura であることを表している。メタ情報の内部表現はユーザ番号であるが、ls コマンドがユーザ名に変換して表示している。

属性の表示方法

ls -l コマンドを用いる。

```
% ls -l a.txt
-rw-r--r-- 1 mura staff 10 May 1 18:18 a.txt
```

グループ staff はファイルがグループ staff に属することを表している。メタ情報の内部表現はグループ番号であるが、ls コマンドがグループ名に変換して表示している。

ファイルサイズ 10 はファイルのサイズが10 バイトであることを表している。

最終変更日時 May 1 18:18 はファイルの最終変更日時である。

パス a.txt はファイルへ到達するために使用したパスである。パス名 (ファイル名) はファイルの属性ではない。

属性の変更方法

chmod コマンドを用いる。

```
% chmod 000 ファイル... # 書式1
% chmod ugo+rwx ファイル... # 書式2
% chmod ugo-rwx ファイル... # 書式3
```

文字	意味
u	所有者 (ユーザ)
g	グループ
o	その他のユーザ
+	権利を与える
-	権利を上げる
r	読出し
w	書込み
x	実行

書式1 000 は3桁の8進数である。8進数で保護モードを指定する。8進数の値のは open システムコールの書式2と同じである。

書式2, 3 ugo+rwx の文字を組合せて保護モードの変更方法を記述する。各文字の意味は上の通りである。例えば、所有者とグループに書込み権と実行権を与える場合なら ug+wx のように書く。その他のユーザの読出し権を取上げるなら o-r のように書く。

属性の変更方法

chmod コマンドの使用例

```
% ls -l a.txt
-rw-r--r-- 1 mura staff 10 May 1 19:42 a.txt
% chmod 640 a.txt
% ls -l a.txt
-rw-r----- 1 mura staff 10 May 1 19:42 a.txt
% chmod g+w a.txt
% ls -l a.txt
-rw-rw---- 1 mura staff 10 May 1 19:42 a.txt
% chmod g-r a.txt
% ls -l a.txt
-rw--w---- 1 mura staff 10 May 1 19:42 a.txt
```

演習 4-3

保護属性に関する演習

- 0440_演習2_リンクと.... 演習.pdf の「2. 保護属性 (rwxrwxrwx) の効果を確認する」をやってみる。

課題 No.3

ファイルシステム

1. 演習 4-1, 4-2, 4-3 を行う。
2. 0440_演習2_リンクと.... 演習.pdf の「3. 1,2の結果を提出する」に従い Github に結果を提出する。

オペレーティングシステムの機能を使ってみよう 第5章 ファイル操作システムコール

ファイル操作システムコール

- ユーティリティコマンド (ln, rm, mkdir, rmdir, chmod ...) 等が使用しているシステムコール。
(コマンドの一覧は「0210_UNIX コマンド初級.pdf」を参照)
- この章では主要な7種類だけ紹介する。
- 以下の内容は macOS 10.13 を基準にしているが、一部では分かり易さのために単純化して説明している場合もある。

unlink システムコール

- ファイル (リンク) を削除するシステムコール。
- ディレクトリの削除には使用できない。
- rm コマンドは、このシステムコールを使用。

書式 path 引数でリンクのパスを一つ指定する。

```
#include <unistd.h>
int unlink(const char *path);
```

解説 unistd.h をインクルードする。
正常時は 0, エラー発生時は -1 を返す。
エラー原因は perror() 関数で表示できる。

使用例 "a.txt" ファイルを削除する例を示す。

```
if (unlink("a.txt") < 0) { // "a.txt" 削除
    perror("a.txt");      // エラー原因表示
    exit(1);              // エラー終了
}
```

mkdir システムコール

- ディレクトリ (フォルダ) 作成するシステムコール。
- mkdir コマンドは、このシステムコールを使用。

書式 path, mode でパスと保護モードを指定する。

```
#include <sys/stat.h>
int mkdir(const char *path, mode_t mode);
```

解説 sys/stat.h をインクルードする必要がある。
正常時は 0, エラー発生時は -1 を返す。
エラー原因は perror() 関数で表示できる。
パスに含まれる途中のディレクトリは作らない。

使用例 "newdir" ディレクトリを rwxr-xr-x で作成する例。

```
if (mkdir("newdir", 0755) < 0) { // "newdir" 作成
    perror("newdir");           // エラー原因
    exit(1);                     // エラー終了
}
```

rmdir システムコール

- ディレクトリを削除するシステムコールである。
- rmdir コマンドは、このシステムコールを利用。
- 空でないディレクトリを削除することはできない。

書式 path 引数でディレクトリのパスを一つ指定する。

```
#include <unistd.h>
int rmdir(const char *path);
```

解説 unistd.h をインクルードする必要がある。
正常時は 0, エラー発生時は -1 を返す。
エラー原因は perror() 関数で表示できる。

使用例 "newdir" と名付けられたディレクトリを削除する例。

```
if (rmdir("newdir") < 0) { // "newdir" 削除
    perror("newdir");      // エラー原因表示
    exit(1);               // エラー終了
}
```

link システムコール (1/2)

- リンク (ハードリンク) を作るシステムコール。
- ln コマンドは、このシステムコールを利用。

書式 存在するパスと新しいパスを指定する。

```
#include <unistd.h>
int link(const char *oldpath,
         const char *newpath);
```

解説 unistd.h をインクルードする必要がある。
正常時は 0, エラー発生時は -1 を返す。
エラー原因は perror() 関数で表示できる。

使用例 1 ファイルにリンク "b.txt" を追加する例。

```
if (link("a.txt", "b.txt") < 0) { // "b.txt" 作成
    perror("link");               // 原因は？
    exit(1);                       // エラー終了
}
```

link システムコール (2/2)

使用例 2 ファイルの移動に応用した例.

```
unlink("b.txt");           // 念のため
if (link("a.txt", "b.txt")<0) { // リンクを作る
    ... エラー処理 ...
}
if (unlink("a.txt")<0) {    // リンクを消す
    ... エラー処理 ...
}
```

オペレーティングシステムの機能を使ってみよ

7/11

symlink システムコール

- シンボリックリンクを作るシステムコール.
- `ln -s` コマンドは、このシステムコールを利用.

書式 シンボリックリンク自身のパスと内容を指定する.

```
#include <unistd.h>
int symlink(const char *path1,
            const char *path2);
```

解説 `unistd.h` をインクルードする必要がある.
正常時は 0, エラー発生時は-1 を返す.
エラー原因は `perror()` 関数で表示できる.

使用例 内容が"a.txt"のシンボリックリンク"b.txt"を作る.

```
if (symlink("a.txt", "b.txt")<0) { // リンクを作る
    perror("b.txt");               // エラー表示
    exit(1);                       // エラー終了
}
```

オペレーティングシステムの機能を使ってみよ

8/11

rename システムコール

- ファイルの移動 (ファイル名の変更) を行うシステムコール.
- `mv` コマンドは、このシステムコールを利用.

書式 新旧二つのパスを指定する.

```
#include <stdio.h>
int rename(const char *from, const char *to);
```

解説 `stdio.h` をインクルードする必要がある.
正常時は 0, エラー発生時は-1 を返す.
エラー原因は `perror()` 関数で表示できる.

引数 `from` は古いパス `to` は移動後の新しいパス.

使用例 "a.txt"のパスを"b.txt"に変更する例.

```
if (rename("a.txt", "b.txt")<0) { // パス名を変更
    perror("rename");             // エラー表示
    exit(1);                     // エラー終了
}
```

オペレーティングシステムの機能を使ってみよ

9/11

chmod (lchmod) システムコール (1/2)

- ファイルの保護モードを変更するシステムコール.
- `chmod` コマンドは、このシステムコールを利用.

書式 パスと保護モード (`rwrxrwx`) を指定する.

```
#include <sys/stat.h>
#include <unistd.h>
int chmod(const char *path, mode_t mode);
int lchmod(const char *path, mode_t mode);
```

解説 `sys/stat.h` をインクルードする必要がある.
シンボリックリンクが指定された場合, `lchmod` はシンボリックリンクの保護モードを変更する.
正常時は 0, エラー発生時は-1 を返す.
エラー原因は `perror()` 関数で表示できる.

オペレーティングシステムの機能を使ってみよ

10/11

chmod (lchmod) システムコール (2/2)

使用例 "a.txt"の保護モードを"`rw-r--r--`"に変更する.

```
if (chmod("a.txt", 0644)<0) { // "rw-r--r--"
    perror("a.txt");           // エラー表示
    exit(1);                   // エラー終了
}
```

オペレーティングシステムの機能を使ってみよ

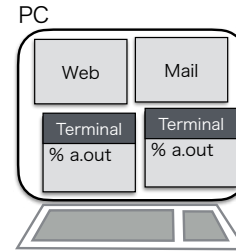
11/11

オペレーティングシステムの機能を使ってみよう 第6章 プロセスとジョブ

オペレーティングシステムの機能を使ってみよう

1 / 15

プロセス



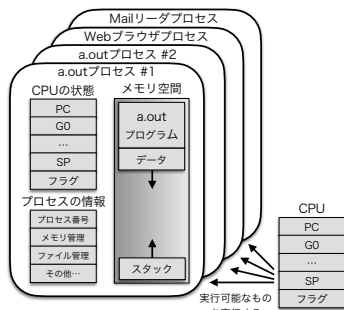
- プログラムは機械語の羅列のこと。
- 同じプログラムが同時に複数実行されることもある。
- 実行中のプログラムのインスタンスをプロセスと呼ぶ。

プロセス = 実行中のプログラム

オペレーティングシステムの機能を使ってみよう

2 / 15

プロセスの構造



- プロセスの情報, CPUの状態 (仮想CPU), メモリ空間 (仮想メモリ)
プロセス = 仮想コンピュータ

オペレーティングシステムの機能を使ってみよう

3 / 15

プロセス関連の UNIX コマンド

ps コマンド (GUIにも似たアプリがある)

- 実行中のプロセスの一覧表を表示するコマンドである。
- 一方のターミナルでvi(vim)を起動し、もう一方のターミナルでpsコマンドを実行した例
- -zsh) は入力されたコマンドを解釈して実行するシェルである。

```
% ps
  PID TTY          TIME CMD
 27828 ttys000    0:00.51  -zsh
 46471 ttys000    0:00.62  vi  chap6s.tex
 38060 ttys001    0:00.10  -zsh
% tty
/dev/ttys001
%
```

- TTY は tty コマンドで確認できる。

```
% tty
/dev/ttys001
%
```

オペレーティングシステムの機能を使ってみよう

4 / 15

プロセス関連の UNIX コマンド

ps コマンドの表示内容

欄	意味
PID	プロセス番号
TTY	制御端末
TT	TTYの簡易表示
TIME	プロセスがこれまでにCPUを使用した時間
CMD	プロセスを起動したコマンド
COMMAND	CMDと同じ
USER	誰の権限で実行しているか
%CPU	CPUの利用率
%MEM	メモリの利用率
VSZ	仮想記憶サイズ (KiB単位)
RSS	常駐セット (KiB単位)
STAT	プロセスの状態
STARTED	プロセスの開始時刻

オペレーティングシステムの機能を使ってみよう

5 / 15

プロセス関連の UNIX コマンド

ps コマンドのオプション

オプション	意味
u	ロングフォーマットで表示 (詳しい表示)
a	他人のプロセスも表示
x	制御端末を持たないものも表示

ps コマンドの実行例 (u オプション)

```
% ps u
USER      PID  %CPU %MEM    VSZ   RSS  TT  STAT STARTED   TIME COMMAND
sigemura 38060  0.5  0.0 408824272 5680 s001  S   11:03AM  0:00.15  -zsh
sigemura 46471  0.0  0.1 408816704 9264 s000  S+  11:33AM  0:00.74  vi
chap6s.tex
sigemura 27828  0.0  0.0 409086416 7472 s000  S   10:35AM  0:00.51  -zsh
```

- u オプションで詳しい表示がされた。

オペレーティングシステムの機能を使ってみよう

6 / 15

プロセス関連の UNIX コマンド

ps コマンドの実行例 (au オプション)

```
% ps au
USER      PID  %CPU %MEM    VSZ   RSS  TT  STAT STARTED   TIME COMMAND
root      60998  0.0  0.0 408766112 1808 s001 R+   12:08PM  0:00.01 ps au
sigemura 46471  0.0  0.1 408816704 9264 s000 S+   11:33AM  0:00.74 vi
chap6s.tex
sigemura 38060  0.0  0.0 408824272 5680 s001 S    11:03AM  0:00.15 -zsh
root      38059  0.0  0.0 408646464 4128 s001 Ss   11:03AM  0:00.02 login -
pfl sigemu
sigemura 27828  0.0  0.0 409086416 7472 s000 S    10:35AM  0:00.51 -zsh
root      27827  0.0  0.0 408646464 4464 s000 Ss   10:35AM  0:00.02 login -
pf sigemur
```

- a オプションで他人のプロセスまで表示された。

プロセス関連の UNIX コマンド

ps コマンドの実行例 (aux オプション)

```
% ps aux
USER      PID  %CPU %MEM    VSZ   RSS  TT  STAT STARTED   TIME COMMAND
WINDOWSERVER
WINDOWSERVER 181 30.5 1.5 410618752 251024 ??  Rs  23Mar23 64:38.93
/System/L
sigemura 43374 10.5 0.6 410251920 97664 ??  R   11:16AM 5:13.83
/Applicat
sigemura 846 4.3 0.6 410068912 102544 ??  S   24Mar23 24:58.94
/System/A
sigemura 12009 3.1 1.5 410249072 245024 ??  S   24Mar23 1:35.29
/System/A
root      418 1.1 0.1 409363184 13936 ??  Rs  23Mar23 3:45.99
/Library/
root      153 1.1 0.1 408268064 16240 ??  Ss  23Mar23 1:28.69
/System/L
...600行程度続く...
%
```

- x オプションで制御端末を持たないプロセスまで表示された。

プロセス関連の UNIX コマンド

ps コマンド STAT 表示の意味

一文字目	意味	二文字目	意味
I	20 秒以上 sleep している	+	フォアグラウンド
S	20 秒未満の sleep	s	セッションリーダー
R	実行可能	...	
T	一時停止状態 (stop, Ctrl-Z)		
Z	ゾンビ (Zombi)		
...			

- 前の実行例の STAT の意味

演習 5-1

CLIでプロセス一覧を見てみよう

- システム内の全プロセスを ps コマンドで表示してみる。
- less コマンドとパイプで接続して表示してみる。
- どんなプロセスが存在するかゆっくり眺める。

(コマンド一覧は「0610_UNIX コマンド (ps など).pdf」参照)

プロセス関連の UNIX コマンド

kill コマンド

kill コマンドの書式

書式

```
kill [-シグナル] PID ...
```

シグナル (省略時は TERM と同じ)

番号	名前	意味
2	INT	終了 (Ctrl-C と同じ)
9	KILL	強制終了
15	TERM	終了 (オプション無しと同じ)
18	TSTP	一時停止 (Ctrl-Z と同じ)
19	CONT	一時停止後の再開

プロセス関連の UNIX コマンド

kill コマンドの使用例

```
% sleep 10000 &                                <--- サンプル用プロセスを起動
[1] 75868
% ps
  PID TTY          TIME CMD
38060 ttys001    0:00.22 -zsh
75868 ttys001    0:00.01 sleep 10000  <--- PIDが分かる
% kill 75868                                     <--- プロセスを終了させる
[1] + terminated sleep 10000
% sleep 10000 &                                <--- 新しいサンプル用プロセスを起動
[1] 75871
% kill -TSTP 75871                               <--- 実はPIDはここでも分かる
[1] + suspended sleep 10000                     <--- プロセスを一時停止
% ps
  PID TTY          TIME CMD
38060 ttys001    0:00.26 -zsh
75871 ttys001    0:00.00 sleep 10000  <--- プロセスは存在している
% kill -CONT 75871                               <--- プロセスを再開させる
% kill 75871                                     <--- プロセスを終了させる
[1] + terminated sleep 10000
```

演習 5-2

CLIでプロセスを操作してみよう

- 前のページの操作を自分のコンピュータで試してみる。
(コマンド一覧は「0610_UNIX コマンド (ps など) .pdf」参照)

オペレーティングシステムの機能を使ってみよ

13 / 15

ジョブ

```
# 通常のコマンド実行
% vi hello.c                                <-- 1プロセスが1ジョブ

# パイプを使用しファイルサイズ順にソートして表示
% ls -l | sort -n --key=5                    <-- 2プロセスが1ジョブ

# 二つのコマンド (ジョブ) を順次実行
% touch a.txt; chmod 777 a.txt               <-- 2ジョブ

# 二つのコマンド (ジョブ) を並列実行
% touch a.txt & touch b.txt                  <-- 2ジョブ
```

フォアグラウンド・ジョブ シェルがジョブの終了を待つ。ジョブが終了したらプロンプトが表示される。

バックグラウンド・ジョブ コマンドの最後に&を付けて実行する。シェルがジョブの終了を待たない。ジョブが終了していてもプロンプトが表示される。次のジョブと並列実行ができる。

オペレーティングシステムの機能を使ってみよ

14 / 15

ジョブ制御

```
% sleep 2000                                <-- フォアグラウンドで起動
^Z
zsh: suspended sleep 2000                   <-- 一時停止した
% bg                                         <-- バックグラウンドで再開
[1] + continued sleep 2000
% sleep 1000 &
[2] 80228
% jobs                                       <-- 実行中のジョブを確認
[1] - running sleep 2000
[2] + running sleep 1000
% fg %1                                     <-- 1番をフォアグラウンドに変更
[1] - running sleep 2000                   <-- フォアグラウンドに変更された
^C
% jobs                                       <-- Ctrl-C で終了
[2] + running sleep 1000                   <-- 2番だけになった
```

Ctrl-C フォアグラウンド・ジョブにINTシグナルを送る。

Ctrl-Z フォアグラウンド・ジョブにTSTPシグナルを送る。

jobs そのシェルが管理しているジョブの一覧を表示する。

fg, bg バックグラウンド・フォアグラウンドの切替え。

オペレーティングシステムの機能を使ってみよ

15 / 15

オペレーティングシステムの機能を使ってみよう 第7章 シグナル

シグナル

前の章で使用して見た kill コマンドや JOB 制御等で使用される。プロセスに非同期的にイベントの発生を知らせる仕組み..

- 1) プロセスや OS がプロセスにイベントを通知
kill コマンド (kill プロセス) が他のプロセスにシグナルを送る。
Ctrl-C や Ctrl-Z が押された時, OS がプロセスにシグナルを送る。
- 2) プロセス自身の異常を通知
0 での割り算 → Floating point exception シグナル (SIGFPE)
ポインタの初期化忘れ → Segmentation fault シグナル (SIGSEGV)
- 3) プロセスが予約した時刻になった
アラームシグナル (SIGALRM)

シグナルの一覧

番号	記号名	デフォルト	説明
1	SIGHUP	終了	プロセスが終了していないときログアウトした。
2	SIGINT	終了	ターミナルで Ctrl-C が押された。
3	SIGQUIT	コアダンプ	ターミナルで Ctrl-\ が押された。
4	SIGILL	コアダンプ	不正な機械語命令を実行した。
8	SIGFPE	コアダンプ	演算でエラーが発生した。
9	SIGKILL	終了	強制終了 (ハンドリングの変更ができない)。
10	SIGBUS	コアダンプ	不正なアドレスをアクセスした場合など。
11	SIGSEGV	コアダンプ	不正なアドレスをアクセスした場合など。
14	SIGALRM	終了	alarm() で指定した時間が経過した。
15	SIGTERM	終了	終了。
17	SIGSTOP	停止	一時停止 (ハンドリングの変更ができない)。
18	SIGTSTP	停止	ターミナルで Ctrl-Z が押された。
19	SIGCONT	無視	一時停止中なら再開する。
20	SIGCHLD	無視	子プロセスの状態が変化した。

- 番号, 記号名, デフォルト (デフォルトのシグナルハンドリング)
- SIGKILL と SIGSTOP はデフォルトから変更できない

シグナルハンドリング

プロセスが, 受信したシグナルをどう扱うか。

- 1) 無視 (*ignore*) そのシグナルを無視する。
- 2) 捕捉・キャッチ (*catch*) そのシグナルを受信し, 登録しておいたシグナル処理ルーチン (シグナルハンドラ関数) を呼び出す。
- 3) デフォルト (*default*) シグナルの種類ごとに決められている初期のハンドリングであり, 以下の四種類のどれかである。各シグナルのデフォルトが四種類のうちのどれかは一覧表から分かる。

停止 プロセスは一時停止状態になる。

無視 プロセスはそのシグナルを無視する。

終了 プロセスは終了する。

コアダンプ プロセスは core ファイルを作成してから終了する。

シグナルの扱い方 = シグナルハンドリング

signal システムコール

自プロセスのシグナルハンドリングを設定する。

書式 sig シグナルの種類, func は新しいハンドリング。

```
#include <signal.h>
// macOS の場合
sig_t signal(int sig, sig_t func);
// Ubuntu Linux の場合
__sighandler_t signal(int sig, __sighandler_t func);
```

解説 sig はハンドリングを変更するシグナル
SIG_IGN はシグナルを無視するようにする。
SIG_DFL はシグナルハンドリングをデフォルトに戻す。
シグナルハンドラ関数を指定すると捕捉になる。
シグナルハンドラ関数のプロトタイプ宣言は次の通り。

```
void func(int sig);
```

戻り値 正常なら変更前のハンドリングが,
エラーなら SIG_ERR が返される。

プログラム例：シグナルを無視する例

SIGINT を一時的に無視するプログラムの例を示す。

```
1 #include <signal.h>
2
3 int main() {
4     ...
5     signal(SIGINT, SIG_IGN); // ここから
6     ...
7     signal(SIGINT, SIG_DFL); // ここまで
8     ...
9 }
```

5行 Ctrl-C を無視するようにハンドリングを変更する。

6行 Ctrl-C を押してもプログラムが終了しない状態。

7行 ハンドリングを元に戻し Ctrl-C で終了するようにする。

プログラム例：シグナルを捕捉する例

SIGINT を捕捉するプログラムの例を示す。

```

1 #include <signal.h>
2
3 void handler(int n) {          // シグナルハンドラ (プロトタイプ宣言どおり)
4     ...                        // シグナル処理
5 }
6
7 int main() {
8     ...
9     signal(SIGINT, handler); // ここから (void f(int)型の関数を引数にする)
10    ...
11    signal(SIGINT, SIG_DFL); // ここまで
12    ...
13 }
```

9行 SIGINT のハンドリングを捕捉にする。

10行 Ctrl-C が押される度に handler() 関数を実行。

11行 SIGINT のハンドリングをデフォルト (終了) に戻す。

オペレーティングシステムの機能を使ってみよ

7/15

シグナルハンドラの制約

1) 制約がある理由

ハンドラ関数はいつ呼ばれるか分からない。

例えば printf() がバッファ操作中にシグナル捕捉！！

ハンドラ関数中で printf() 実行 → 何か悪いことが起こる

2) やってもよいこと

1. シグナルハンドラ関数のローカル変数の操作

2. volatile sig_atomic_t 型変数の読み書き

sig_atomic_t 型は macOS では int 型

(「読み書き」は単純な参照と代入のことだけ指す)

volatile を付けると C コンパイラの最適化の対象外

(最適化は非同期にアクセスを前提にしていない。)

3. 非同期シグナル安全な関数の呼び出し

_exit(), alarm(), chdir(), chmod(), close(), creat(),

dup(), dup2(), execl(), execve(), fork(), kill(), ...

オペレーティングシステムの機能を使ってみよ

8/15

シグナルハンドラの例

```

1 #include <unistd.h>
2 #include <signal.h>
3 volatile sig_atomic_t flg = 0; // シグナルハンドラが操作しても良い
4 void handler(int n) {
5     flg = 1; // 単純な代入
6     write(1, "Ctrl-C\n", 7); // 非同期シグナル安全な関数の実行
7 }
8 int main(int argc, char **argv) {
9     int cnt = 0;
10    signal(SIGINT, handler);
11    while (cnt < 3) {
12        if (flg) { // 単純な参照
13            cnt++;
14            flg = 0; // 単純な代入
15        }
16    }
17    return 0;
18 }
```

3行 ハンドラ関数が操作できる変数 flg を宣言

4行 ハンドラ関数 (限られた操作しかできない)

オペレーティングシステムの機能を使ってみよ

9/15

kill システムコール

プロセスがプロセスにシグナルを送信するシステムコール

書式 pid は送り先プロセス, sig はシグナルの種類

```
#include <signal.h>
```

```
int kill(pid_t pid, int sig);
```

解説 送信先のプロセスとシグナルの種類を指定してシグナルを送信する。(シグナルの種類は前出の表の通り)

戻り値 正常時は 0, 異常時-1 が返される。

プログラム例 kill システムコールの使用例として, kill コマンドを簡単化したプログラム (mykill) を示す。

使用方法: mykill <シグナル番号> <プロセス番号>

オペレーティングシステムの機能を使ってみよ

10/15

mykill プログラム

```

1 #include <stdio.h>
2 #include <stdlib.h> // atoi のために必要
3 #include <signal.h> // kill のために必要
4
5 int main(int argc, char *argv[]) {
6     if (argc != 3) {
7         fprintf(stderr, "Usage : %s SIG PID\n", argv[0]);
8         return 1;
9     }
10
11    int sig = atoi(argv[1]); // 第1引数
12    int pid = atoi(argv[2]); // 第2引数
13
14    if (kill(pid, sig) < 0) {
15        perror(argv[0]);
16        return 1;
17    }
18    return 0;
19 }
```

- atoi() は, 文字列"123"を整数 123 に変換

オペレーティングシステムの機能を使ってみよ

11/15

mykill の実行例

```

% ./mykill                               <-- 使い方が分からない
Usage : ./mykill SIG PID                 <-- 使い方を表示してくれる
% sleep 1000 &                           <-- sleep が JOB=1, PID=13589
[1] 13589                                  <-- だと分かる
% ./mykill 2 13589                         <-- PID=13589のプロセスにSIGINT
(2)を送る
[1] + interrupt sleep 1000
% ./mykill 100 13589
./mykill: Invalid argument                 <-- シグナル番号が不正
% ./mykill 2 13589
./mykill: No such process                  <-- プロセス番号が不正
%
```

- sleep を使用する実行例

オペレーティングシステムの機能を使ってみよ

12/15

sleep システムコール

自プロセスを指定された時間、待ち状態にする。

書式 seconds は待ち時間（秒単位）

```
#include <unistd.h>
unsigned int sleep(unsigned int seconds);
```

解説 決められた時間待ち状態になる。途中でシグナルが届いた場合は終了する。（ハンドリングが無視以外の場合）

戻り値 sleep 予定だった残りの秒数が返される。（通常は0のはず）

プログラム例 1秒に一度helloと表示するプログラム（Ctrl-Cで終了）

```
#include <stdio.h>
#include <unistd.h>
int main() {
    for (;;) {
        printf("hello\n"); // hello表示
        sleep(1);          // 1秒待つ
    }
    return 0;
}
```

オペレーティングシステムの機能を使ってみよう

13 / 15

pause システムコール

自プロセスを待ち状態にする（時間制限なし）。

```
#include <unistd.h>
int pause(void);
```

解説 時間を定めず待ち状態になる。途中でシグナルが届いた場合は終了する。（ハンドリングが無視以外の場合）

戻り値 常にエラーで終了するので-1

プログラム例 SIGINTのハンドリングを捕捉にした例

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
void handler(int n) {} // 何もしないハンドラ関数
int main() {
    signal(SIGINT, handler); // SIGINTを捕捉に変更する
    pause();                 // 1回目の Ctrl-C を待つ
    pause();                 // 2回目の Ctrl-C を待つ
    pause();                 // 3回目の Ctrl-C を待つ
    return 0;                // Ctrl-C 3回で終了する
}
```

オペレーティングシステムの機能を使ってみよう

14 / 15

alarm システムコール

アラームシグナルの発生を予約する。

```
#include <unistd.h>
unsigned int alarm(unsigned int seconds);
```

解説 seconds 秒後に SIGALRMが発生する。
予約するだけでプロセスが待ち状態になるわけではない。

戻り値 前回の alarm システムコールの残り時間（通常0）

プログラム例 SIGALRMのハンドリングを捕捉にした例

```
#include <stdio.h>
#include <unistd.h> // alarm, pause のために必要
#include <signal.h> // signal のために必要
void handler(int n) {} // 何もしないハンドラ関数
int main(int argc, char *argv[]) {
    signal(SIGALRM, handler); // SIGALRMを捕捉に変更する
    alarm(3);
    printf("pause()します\n");
    pause(); // プロセスが停止する
    printf("pause()が終わりました\n");
    return 0;
}
```

オペレーティングシステムの機能を使ってみよう

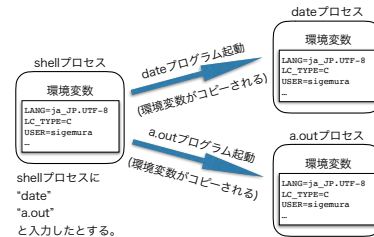
15 / 15

オペレーティングシステムの機能を使ってみよう 第 8 章 環境変数

オペレーティングシステムの機能を使ってみよう

1 / 24

環境変数



- シェルが管理する変数
- シェルからプログラムにコピーされる。
- プログラムは実行時に環境変数の値を調べることができる。
- 同じプログラムで複数の言語に対応すること等ができる。

オペレーティングシステムの機能を使ってみよう

2 / 24

環境変数と使用例 (1)

macOS や UNIX でよく使用される環境変数

```

SHELL=/bin/zsh          # 使用中のシェル
TERM=xterm-256color     # 使用中のターミナルエミュレータ
USER=sigemura           # 現在のユーザー
PATH=/usr/bin:/bin:/usr... # シェルがコマンドを探すディレクトリ一覧
PWD=/Users/sigemura     # カレントディレクトリのパス
HOME=/Users/sigemura    # ユーザーのホームディレクトリ
LANG=ja_JP.UTF-8        # ユーザーが使用したい言語 (ja_JP.UTF-8 (日本語))
LC_TIME=C               # ユーザーが日時の表示に使用したい言語 (C言語標準)
TZ=Japan                # どの地域の時刻を使用するか (日本)
CLICOLOR=1              # ls コマンド等がカラー出力する (yes)
  
```

- 本当はもっとたくさんの環境変数がある。
- ここでは「名前=値」形式で一覧を表示している。
- 次頁は LC_TIME 環境変数と TZ 環境変数を変更して実行した例

オペレーティングシステムの機能を使ってみよう

3 / 24

環境変数と使用例 (2)

```

% export LC_TIME=C      # LC_TIME環境変数を作ってC言語標準 (米国英語) を表示値をセット
% date                 # 英語表記, 日本時間の現在時刻
Sun Apr  2 08:15:20 JST 2023
% ls -l Makefile      # 日本語表記, 日本時間の現在時刻を表示する
-rw-r--r--  1 sigemura  staff  128 Apr  1 10:40 Makefile
% LC_TIME=ja_JP.UTF-8  # LC_TIMEに日本語表記を表示値をセットして試す
% date                 # 日本語表記, 日本時間の現在時刻を表示する
2023年 4月 2日 日曜日 08時16分44秒 JST
% ls -l Makefile      # 日本語表記, キューバ時間を表示値をセット
-rw-r--r--  1 sigemura  staff  128  4  1 10:40 Makefile
% export TZ=Cuba       # TZ環境変数を作ってキューバ時間を表示値をセット
% date                 # 日本語表記, キューバ時間の現在時刻
2023年 4月 1日 土曜日 19時19分50秒 CDT
% ls -l Makefile      # 日本語表記, キューバ時間を表示値をセット
-rw-r--r--  1 sigemura  staff  128  3 31 21:40 Makefile
%
  
```

- LC_TIME 環境変数は日時の表示形式を決める。
- TZ 環境変数はどの地域の時刻を表示するか決める。

オペレーティングシステムの機能を使ってみよう

4 / 24

環境変数を誰が決めるか

- (1) システム管理者
システム管理者はユーザーがログインした時の初期状態を決める。
UNIX や macOS では管理者が作成したスクリプトが初期化を行う。
管理者は全ユーザーに共通の初期化処理をここ (/etc/zprofile) に書いておく。
- (2) ユーザーの設定ファイル
ユーザーは自分のホームディレクトリのファイルに初期化手順を書く。
初期化スクリプト (.zprofile) の例を示す。

```

PATH="/usr/local/bin:$PATH:$HOME/bin:."
export LC_TIME=C
export CLICOLOR=1
      
```
- (3) ユーザによるコマンド操作
シェルのコマンド操作で環境変数を操作することができる。
影響範囲は操作したウィンドのシェルのみである。
次のログイン時には操作結果の影響は残らない。

オペレーティングシステムの機能を使ってみよう

5 / 24

環境変数の操作 (1)

環境変数を表示するコマンド (printenv)

書式 name は環境変数の名前である。

```
printenv [name]
```

解説 name を省略した場合は、全ての環境変数の名前と値を表示する。
name を書いた場合は該当のする環境変数の値だけ表示する。
該当する環境変数がない場合は何も表示しない。

実行例 macOS 上での printenv コマンドの実行例を示す。
環境変数の名前を省略して実行した場合は、全ての環境変数について「名前=値」形式で表示される。

```

% printenv
SHELL=/bin/zsh          <--- 「名前=値」形式で表示
TERM=xterm-256color
USER=sigemura
...
% printenv SHELL        <--- SHELL環境変数を表示する
/bin/zsh                ('値' だけ表示される)
% printenv NEVER        <--- 何も表示されない
%
  
```

オペレーティングシステムの機能を使ってみよう

6 / 24

環境変数の操作 (2)

環境変数を新規作成する手順 (その1) - sh の場合 -

書式 次の2ステップで操作を行う。

```
name=value
export name
```

解説 1行で、一旦、シェル変数を作る。
2行でシェル変数を環境変数に変更する。

実行例 1行は MYNAME 環境変数が存在するか確認している。
(MYNAME 環境変数は存在しないので何も表示されない。)
2, 3行で値が sigemura の MYNAME 環境変数を作った。
4行で MYNAME 環境変数を確認する。
(値が sigemura になっていることが分かる。)

```
1 % printenv MYNAME          <--- MYNAMEは存在しない
2 % MYNAME=sigemura         <--- シェル変数MYNAMEを作る
3 % export MYNAME            <--- MYNAMEを環境変数に変更する
4 % printenv MYNAME
sigemura                    <--- 環境変数MYNAMEの値
5 %
```

オペレーティングシステムの機能を使ってみよ

7 / 24

環境変数の操作 (3)

環境変数を新規作成する手順 (その2) - zsh の場合 -

書式 次の1ステップで環境変数を作ることができる。

```
export name=value
```

解説 一旦、シェル変数を作ることなく環境変数を作ることができる。

実行例 次のように動作確認ができる。

```
% printenv MYNAME          <--- MYNAME環境変数は存在しない
% export MYNAME=sigemura
% printenv MYNAME          <--- MYNAME環境変数ができていた
sigemura
%
```

オペレーティングシステムの機能を使ってみよ

8 / 24

環境変数の操作 (4)

環境変数の値を変更する手順

書式 name は環境変数の名前, value は新しい値である。

```
name=value
```

解説 「環境変数の変更」と「シェル変数の作成」は書式だけでは区別が付かない、変数名を間違った場合、間違った名前で新しいシェル変数が作成されエラーにならないので注意が必要である。

実行例 MYNAME 環境変数が既に存在している場合の実行例を示す。

```
% printenv MYNAME          <--- 値を表示する
sigemura
% MYNAME=tetsuji           <--- 値を変更する
% printenv MYNAME
tetsuji                    <--- 変更されている
%
```

オペレーティングシステムの機能を使ってみよ

9 / 24

環境変数の操作 (5)

環境変数の値を参照する手順 (1)

書式 name は環境変数の名前である。

```
$name
```

解説 \$name は変数の値に置き換えられる。

実行例 1 PATH 環境変数の値にディレクトリを追加する例。

```
% printenv PATH            # PATH の初期値を確認
/bin:/usr/bin
% PATH=$PATH:.             # カレントディレクトリを追加
% printenv PATH
/bin:/usr/bin:.
% PATH=$PATH:$HOME/bin     # ホームのbinを追加
% printenv PATH
/bin:/usr/bin:./:/User/sigemura/bin
%
```

オペレーティングシステムの機能を使ってみよ

10 / 24

環境変数の操作 (6)

環境変数の値を参照する手順 (2)

実行例 2 環境変数 i の値をインクリメントする例。

```
% export i=1               # 環境変数 i を作る
% printenv i
1
% i=`expr $i + 1`          # クォートはバッククォート
% echo $i
2
%
```

- expr は式の計算結果を表示するコマンド。
- バッククォートの内部は実行結果と置き換わる。
- printenv i の代わりに echo \$i でも値を表示できる。

オペレーティングシステムの機能を使ってみよ

11 / 24

環境変数の操作 (7)

環境変数を削除する手順

書式 name は変数の名前である。

```
unset name
```

解説 存在しない変数を unset してもエラーにならない。
変数名を間違ってもエラーにならないので注意が必要である。

実行例 MYNAME 環境変数が既に存在している場合の実行例を示す。

```
% printenv MYNAME
tetsuji
% unset MYNAME
% printenv MYNAME
# MYNAMEは存在しない
%
```

オペレーティングシステムの機能を使ってみよ

12 / 24

環境変数の操作 (8)

env コマンドを用いて環境変数を一時的に変更する手順

書式 変数=値を代入が続いた後にコマンドが続く。

```
env name1=value1 name2=value2 ... command
```

解説 最初の代入形式を環境変数の変更 (作成) 指示とみなす。代入形式ではないものを以降で実行すべきコマンドとみなす。

実行例 ロケールとタイムゾーンを変更して date を実行する。
LC_TIME 環境変数は日時表示用のロケールを格納する。
TZ 変数はタイムゾーンを格納する。

```
% date
Sun Apr  2 08:56:40 JST 2023      <--- 普通は日本時間、英語表記
% env LC_TIME=ja_JP.UTF-8 TZ=Cuba date
2023年 4月 1日 土曜日 19時56分55秒 CDT  <--- キューバ時間、日本語表記
% date
Sun Apr  2 08:57:00 JST 2023      <--- 後のコマンドに影響はない
%
```

オペレーティングシステムの機能を使ってみよう

13 / 24

ロケール (ユーザの言語や地域を定義する)

LANG 環境変数や LC_TIME 環境変数にセットする値をロケール名と呼ぶ。ロケール名は次の組み合わせで表現される。
「言語コード」、「国名コード」、「エンコーディング」

- 言語コードは ISO639 で定義された 2 文字コードである。(日本語は"ja")
- 国名コードは ISO3166 で定義された 2 文字コードである。(日本は"JP")
- エンコーディングは、使用する文字符号化方式を示す。(macOS や Linux では UTF-8 方式が使用される。)
- 使用可能なロケールの一覧は locale -a コマンドで表示できる。

macOS で日本語を使用する場合のロケール名は次の通り。
ja_JP.UTF-8 (日本語_日本.UTF-8)

オペレーティングシステムの機能を使ってみよう

14 / 24

タイムゾーン (時差が同じ地域)

どの地域の時間で時刻を表示するかを環境変数で制御できる。

- 日本時間は協定世界時 (UTC) と時差がマイナス 9 時間
- TZ 環境変数にタイムゾーンを表す値をセットする。
- OS の内部の時刻は協定世界時 (UTC)
- 時刻を表示する時に TZ を参照して現地時間に変換する。
- 日本時間は TZ=JST-9 となる。
 - /usr/share/zoneinfo/ディレクトリのファイル名でも指定できる。
 - Cuba ファイルが存在するので TZ=Cuba と指定できる。
 - Japan ファイルも存在するので TZ=Japan も指定できる。
 - Asia/Tokyo ファイルが存在するので TZ=Asia/Tokyo も可。

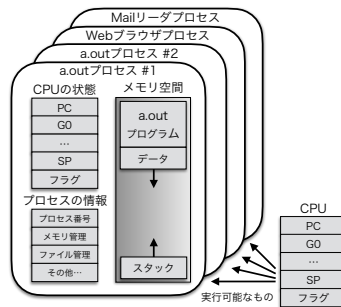
TZ 環境変数が定義されていない時は、OS のインストール時に選択した標準のタイムゾーンが用いられる。

オペレーティングシステムの機能を使ってみよう

15 / 24

環境変数の仕組み (0)

参考: プロセスの構造 (6 章で紹介したもの)

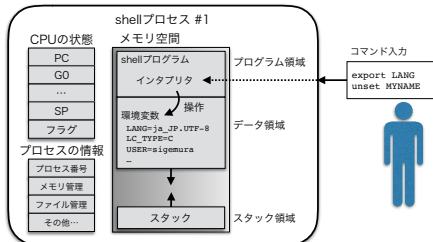


オペレーティングシステムの機能を使ってみよう

16 / 24

環境変数の仕組み (1)

シェルによる管理



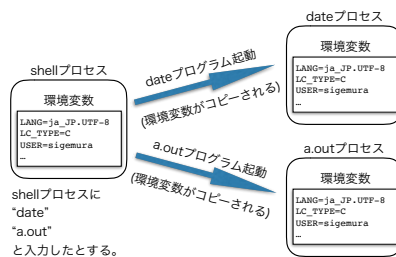
- 環境変数はシェルのプロセスのメモリ空間に記憶されている。
- コマンドが入力されるとシェルのインタプリタが意味を解釈する。
- 環境変数を操作するコマンドならメモリ空間を操作する。
- 環境変数を操作するコマンドは内部コマンド

オペレーティングシステムの機能を使ってみよう

17 / 24

環境変数の仕組み (2)

プロセスへのコピー



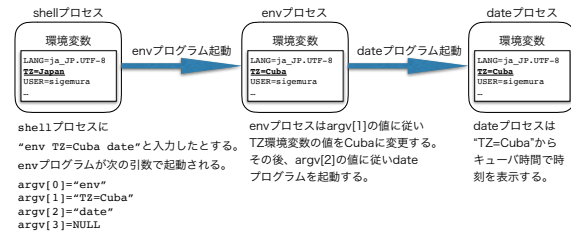
- シェルは子プロセスとして外部コマンドを起動する。
- 外部コマンドの起動時に子プロセスに環境変数をコピーする。
- 子プロセスはコピーされた環境変数を参照・変更・削除できる。

オペレーティングシステムの機能を使ってみよう

18 / 24

環境変数の仕組み (3)

変更した上でのコピー



- 他のプログラムを起動する時に環境変数をコピーする。
- env コマンドは他のプログラムを起動するプログラムの例。
 1. env コマンドは自身の環境変数を変更する。
 2. env コマンドは指定されたプログラムを起動する。
 3. env コマンドは、この際、環境変数をコピーする。

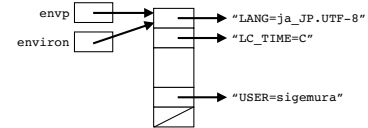
オペレーティングシステムの機能を使ってみよ

19 / 24

プログラムからの環境変数アクセス (1)

読み出し (envp 仮引数, environ 変数を用いる)

データ構造 メモリ内で環境変数は次のようなデータ構造



プログラム例 全ての環境変数を name=val 形式で印刷する。

```

1 #include <stdio.h>
2 extern char **environ;           // 外部で定義されている
3 int main(int argc, char *argv[]) { // 今回はenvpは不要
4     for (int i=0; environ[i]!=NULL; i++) { // NULLが見つかるまで
5         printf("%s\n", environ[i]);       // 環境変数を印刷
6     }
7     return 0;                     // 必ず正常終了
8 }

```

オペレーティングシステムの機能を使ってみよ

20 / 24

プログラムからの環境変数アクセス (2)

読み出し (getenv 関数を用いる)

書式 getenv 関数に変数名を与えると値が返る。

```

#include <stdlib.h>
char *getenv(const char *name);

```

プログラム例 LANG 環境変数の値を表示する。

```

1 #include <stdio.h>
2 #include <stdlib.h>           // getenv() のために必要
3 int main(int argc, char *argv[]) {
4     char *val = getenv("LANG"); // LANG環境変数の値を調べる
5     if (val!=NULL) {           // 見つかったら
6         printf("LANG=%s\n", val); // 値を表示
7     } else {                   // 見つからない時は
8         printf("LANG does not exist.\n"); // エラーメッセージを表示
9     }
10    return 0;                  // 正常終了
11 }
12 /* 実行例
13 % ./a.out
14 LANG=ja_JP.UTF-8
15 */

```

オペレーティングシステムの機能を使ってみよ

21 / 24

プログラムからの環境変数アクセス (3)

作成と値の変更 (setenv 関数)

書式 変数名 (name), 値 (val), フラグ (overwrite) を与える。

```

#include <stdlib.h>
int setenv(const char *name, const char *val, int overwrite);

```

解説 `overwrite=0` で上書き禁止になる。
 返り値は、正常時 0, エラー時 -1 である。
 上書き禁止時、既に変数が存在するとエラーになる。

使用例 MYNAME 環境変数の値を sigemura にする。

```
setenv("MYNAME", "sigemura", 1);
```

この例は上書き許可の場合。

オペレーティングシステムの機能を使ってみよ

22 / 24

プログラムからの環境変数アクセス (4)

作成と値の変更 (putenv 関数)

書式 name=val 形式の文字列 (string) を与える。

```

#include <stdlib.h>
int putenv(char *string);

```

解説 `name=val` 形式以外の文字列を与えるとエラーになる。
 返り値は正常時 0, エラー時 -1 である。
`putenv` 関数は常に上書き許可になる。

使用例 MYNAME 環境変数の値を sigemura にする。

```
putenv("MYNAME=sigemura");
```

次の setenv と同じ。

```
setenv("MYNAME", "sigemura", 1);
```

オペレーティングシステムの機能を使ってみよ

23 / 24

プログラムからの環境変数アクセス (5)

削除 (unsetenv 関数)

書式 削除する変数の名前 (name) を与える。

```

#include <stdlib.h>
int unsetenv(const char *name);

```

解説 名前 (name) を指定して環境変数を削除する。
 エラー時 -1 が返る、正常時は 0 が返る。

使用例 MYNAME 環境変数を削除する。

```
unsetenv("MYNAME");
```

オペレーティングシステムの機能を使ってみよ

24 / 24

オペレーティングシステムの機能を使ってみよう 第9章 プロセスの生成とプログラムの実行

オペレーティングシステムの機能を使ってみよう

1 / 24

spawn 方式と fork-exec 方式

新しいプログラムを実行する方式は次の二種類がある。

- **spawn 方式** (スプーン：卵を産む方式)
Windows 等で使用されてきた。
次の3ステップを一つのシステムコールで行う。
 1. プロセスを作る。
 2. プロセスを初期化する。
 3. プログラムを実行する。
- **fork-exec 方式** (分岐-実行方式)
UNIX 系の OS で使用されてきた。
次の3ステップを二つのシステムコールとプログラムで行う。
 1. プロセスを作る (fork システムコール)。
 2. プロセスを初期化する (ユーザプログラム)。
 3. プログラムを実行する (exec システムコール)。

オペレーティングシステムの機能を使ってみよう

2 / 24

spawn 方式

posix_spawn の例

書式 次の通りである。

```
#include <spawn.h>
int posix_spawn(pid_t *pid, const char *path,
               const posix_spawn_file_actions_t *file_actions,
               const posix_spawnattr_t *attrp,
               char *const argv[], char *const envp[]);
```

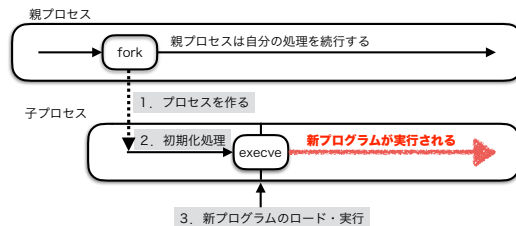
解説 新しいプロセスを作り path で指定したプログラムを実行する。

引数 pid は新しいプロセスのプロセス番号を格納する変数を指すポインタ。path は実行するプログラムを格納したファイルのパスである。絶対パスでも相対パスでも良い。file_actions, attrp はプロセスの初期化を指示するデータ構造へのポインタ。argv, envp は実行されるプログラムに渡すコマンド行引数と環境変数である。

オペレーティングシステムの機能を使ってみよう

3 / 24

fork-exec 方式 (1)



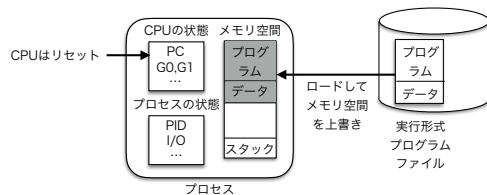
1. 新しいプロセス (子プロセス) を作る (fork システムコール)。
2. ユーザプログラムに従い子プロセスが自ら初期化処理を行う。
3. 新しいプログラムをロード・実行 (execve システムコール) する。
プログラムで初期化処理を行うので柔軟性が高い。

オペレーティングシステムの機能を使ってみよう

4 / 24

fork-exec 方式 (2)

プログラムのロードと実行 (execve システムコール) の概要



- プロセスのメモリ空間に新しいプログラムをロードする。
- execve システムコールを発行したプログラムは上書きされて消える。
- プロセスの仮想 CPU はリセットされプログラムの先頭から実行。
- プロセスが新しいプログラムに変身した。

オペレーティングシステムの機能を使ってみよう

5 / 24

fork-exec 方式 (3)

execve システムコール

書式 execve システムコールの書式は次の通りである。

```
#include <unistd.h>
int execve(const char *path,
           char *const argv[], char *const envp[]);
```

解説 自プロセスで path で指定したプログラムを実行する。正常時には execve を実行したプログラムは新しいプログラムで上書きされ消える。execve システムコールが戻る (次の行が実行される) のはエラー発生時だけである。

引数 path は実行するプログラムを格納したファイルのパスである。絶対パスでも相対パスでも良い。argv, envp は新しいプログラムに渡すコマンド行引数と環境変数である。

オペレーティングシステムの機能を使ってみよう

6 / 24

fork-exec 方式 (4)

execve システムコールの使用例 1

```
#include <stdio.h>          // perror のために必要
#include <unistd.h>          // execve のために必要
extern char **environ;
char *args[] = { "date", NULL };
char *execpath="/bin/date";
int main(int argc, char *argv[], char *envp[]) {
    execve(execpath, args, environ); // /bin/dateを自分と同じ環境変数で実行
    perror(execpath);               // ※ execveが戻ってきたらエラー！
    return 1;
}
/* 実行例 (英語表示、日本時間で表示される)
% ./executest1
Sun Apr  2 09:20:24 JST 2023
*/
```

- /bin/date プログラムをロード・実行する。
- date プログラムの argv 配列を準備して渡す。
- 環境変数は自身のものを渡す。
- execve が戻ってきたら無条件にエラー処理をする。

オペレーティングシステムの機能を使ってみよう

7 / 24

fork-exec 方式 (5)

execve システムコールの使用例 2

```
#include <stdio.h>          // perror のために必要
#include <unistd.h>          // execve のために必要
#include <stdlib.h>          // putenv のために必要
extern char **environ;
char *args[] = { "date", NULL };
char *execpath="/bin/date";
int main(int argc, char *argv[], char *envp[]) {
    putenv("LC_TIME=ja_JP.UTF-8"); // 自分の環境変数を変更する
    execve(execpath, args, environ); // /bin/dateを自分と同じ環境変数で実行
    perror(execpath);               // execveが戻ってきたらエラー！
    return 1;
}
/* 実行例 (日本語表示、日本時間で表示される)
% ./executest3
2023年 4月 2日 日曜日 09時24分29秒 JST
*/
```

- (環境変数を変更＝初期化処理) をした上で execve をする。
- putenv() 関数を用いて自身の環境変数を書き換え。
- execve に自身の環境変数を渡す。

オペレーティングシステムの機能を使ってみよう

8 / 24

fork-exec 方式 (6)

execve システムコールの使用例 3

```
#include <stdio.h>          // perror のために必要
#include <unistd.h>          // execve のために必要
char *args[] = { "date", NULL };
char *envs[] = { "LC_TIME=ja_JP.UTF-8", "TZ=Cuba", NULL };
char *execpath="/bin/date";
int main(int argc, char *argv[], char *envp[]) {
    execve(execpath, args, envs); // /bin/dateを上記の環境変数で実行
    perror(execpath);             // execveが戻ってきたらエラー！
    return 1;
}
/* 実行例 (日本語表示、キューバ時間で表示される)
% ./executest2
2023年 4月 1日 土曜日 20時25分52秒 CDT
*/
```

- 全く新しい環境変数の一覧を渡す例。
- date プログラムが必要とする環境変数だけの配列を渡す。

オペレーティングシステムの機能を使ってみよう

9 / 24

fork-exec 方式 (7)

execve システムコールの使用例 4

```
#include <stdio.h>          // perror のために必要
#include <unistd.h>          // execve のために必要
extern char **environ;
char *args[] = { "echo", "aaa", "bbb", NULL }; // "% echo aaa bbb" に相当
char *execpath="/bin/echo";
int main(int argc, char *argv[], char *envp[]) {
    execve(execpath, args, environ); // /bin/echoを自分と同じ環境変数で実行
    perror(execpath);               // execveが戻ってきたらエラー！
    return 1;
}
/* 実行例
% ./executest4
aaa bbb <--- /bin/echo の出力
*/
```

- 複数のコマンド行引数をもつプログラム (echo) の実行例。
- argv[0] にプログラムの名前を入れ忘れないように。

オペレーティングシステムの機能を使ってみよう

10 / 24

fork-exec 方式 (8)

execve システムコールのラッパー関数

- 関数の内部で execve システムコールを発行 (wrapper)

書式 四つのラッパー関数の書式をまとめて掲載

```
#include <unistd.h>
int execl(const char *path, char *const argv[]);
int execlp(const char *path,
            const char *argv0, ... , *argvn, NULL);
int execlp(const char *file,
            const char *argv0, ... , *argvn, NULL);
```

意味 execl("/bin/date", argv);
→ execve("/bin/date", argv, environ);
execlp("/bin/date", argv, environ);
execl("/bin/echo", "echo", "aaa", "bbb", NULL);
execlp("echo", "echo", "aaa", "bbb", NULL);

オペレーティングシステムの機能を使ってみよう

11 / 24

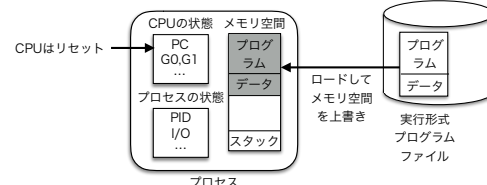
fork-exec 方式 (9)

入出力のリダイレクト 1

- リダイレクトの復習

```
% echo aaa bbb <--- echoの出力が表示される
aaa bbb
% echo aaa bbb > a.txt <--- echoの出力が表示されない
% cat a.txt
aaa bbb <--- echoの出力がa.txtに格納されてた
```

- プログラムは標準入出力を自らオープンする必要が無かった。
- プロセスの状態が execve 前のプログラムから引き継がれるから。



オペレーティングシステムの機能を使ってみよう

12 / 24

fork-exec 方式 (10)

入出力のリダイレクト 2

- リダイレクトはプログラムのロード・実行前にシェルが行う。
- シェルが標準入出力をファイルに接続してから `execve` している。
- 原理を表すプログラム例

```
#include <stdio.h> // perror のために必要
#include <unistd.h> // execl のために必要
#include <fcntl.h> // open のために必要
char *execpath="/bin/echo";
char *outfile="aaa.txt";
int main(int argc, char *argv[], char *envp[]) {
    close(1); // 標準出力をクローズする
    int fd = open(outfile,
        O_WRONLY|O_CREAT|O_TRUNC, 0644); // 標準出力をオープンしなおす
    if (fd!=1) { // 標準出力以外になっている
        fprintf(stderr, "something is wrong\n"); // 原因が分からないが...
        return 1; // 何か変なのでエラー終了
    }
    execl(execpath, "echo", "aaa", "bbb", NULL); // /bin/echo を実行
    perror(execpath); // execl が戻ったらエラー！
    return 1;
}
```

オペレーティングシステムの機能を使ってみよ

13 / 24

課題 No.9

1. 前のページ (リスト 9.5) のプログラムを入力し実行してみる。
2. 入力のリダイレクトをするプログラム例を作る。
ヒント：標準入力のファイルディスクリプタは 0 番である。
3. `env` コマンドのクローン `myenv`
`putenv()` がエラーになるまでコマンド行引数を環境変数の設定と
思っ使う。残りが実行するコマンドを表している。下の実行例では
`putenv(argv[1]);`
`putenv(argv[2]);`
`putenv(argv[3]);`
`execvp(argv[3], &argv[3]);`
(三回目の `putenv()` はエラーになる)
が実行されるようにプログラムを作る。

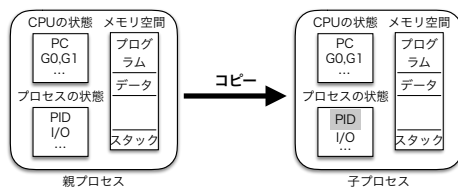
```
% ./myenv LC_TIME=ja_JP.UTF-8 TZ=Cuba ls -l
```

オペレーティングシステムの機能を使ってみよ

14 / 24

fork-exec 方式 (11)

新しいプロセスを作る (fork システムコール) 1



- fork システムコールはプロセスのコピー分身を作る。
- もとととのプロセスが親プロセス、分身が子プロセス。
- 分身は PID 以外は同じ (CPU の PC も同じ)。
- 子プロセスは fork システムコールの途中から実行を開始する。

オペレーティングシステムの機能を使ってみよ

15 / 24

fork-exec 方式 (12)

新しいプロセスを作る (fork システムコール) 2

書式 fork の書式を示す。

```
#include <unistd.h>
int fork(void);
```

解説 fork システムコールが終了する際、親プロセスには子プロセスの PID が返され、子プロセスには 0 が返される。プログラムはこの値を目印に自分が親か子か判断できる。エラー時は、親プロセスに -1 が返され子プロセスは作られない。

```
int main() {
    int x = 10;
    int pid;
    pid = fork(); // この瞬間にプロセスがコピーされる
    if (pid<0) {
        fprintf(stderr, "forkでエラー発生\n"); // エラーの場合
        return 1;
    } else if (pid!=0) { // 親プロセスだけが以下を実行する
        x = 20; // 親プロセスの x を書き換える
    }
}
```

オペレーティングシステムの機能を使ってみよ

16 / 24

fork-exec 方式 (13)

新しいプロセスを作る (fork システムコール) 3

使用例 親プロセスと子プロセスが並行実行される状態になる。

```
1 #include <stdio.h> // printf, fprintf のために必要
2 #include <unistd.h> // fork のために必要
3 int main() {
4     int x = 10;
5     int pid;
6     pid = fork(); // この瞬間にプロセスがコピーされる
7     if (pid<0) {
8         fprintf(stderr, "forkでエラー発生\n"); // エラーの場合
9         return 1;
10    } else if (pid!=0) { // 親プロセスだけが以下を実行する
11        x = 20; // 親プロセスの x を書き換える
12        printf("親 pid=%d x=%d\n", pid, x);
13    } else { // 子プロセスだけが以下を実行する
14        printf("子 pid=%d x=%d\n", pid, x); // 子プロセスの x は初期値のまま
15    }
16    return 0;
17 }
```

オペレーティングシステムの機能を使ってみよ

17 / 24

fork-exec 方式 (14)

プロセスの終了と待ち合わせ

例えば次のような手順で処理がされる。

- 親プロセスは子プロセスをいくつか作成し、それらに同時に並行して処理を行わせる。
- 子プロセスは処理を終えると終了する。
- 子プロセスが処理を終えると、親プロセスは子プロセスが正常に終了したかチェックする。

% ls -l | grep rwx ← 二つのプロセスが並列実行される
- 子プロセスが処理結果と共に自身を終了する。
→ `exit` システムコール
- 親プロセスが子プロセスの終了を待つ。
→ `wait` システムコール

オペレーティングシステムの機能を使ってみよ

18 / 24

fork-exec 方式 (15)

exit システムコール：自プロセスを終了する。

書式 exit システムコールの書式を示す。

```
#include <stdlib.h>
void exit(int status);
```

解説

- プロセスが終了するので exit は戻らない。
- status はプロセスの終了ステータスである。
(0 ≤ status ≤ 255)
- 親プロセスは wait で終了ステータスを受け取る。
- C プログラムの main 関数は exit の引数で実行。
exit(main(argc, argv, envp));
- 次の C プログラムは同じ結果になる。

```
int main() {
    ...
    exit(1);
    ...
}

=

int main() {
    ...
    return 1;
    ...
}
```

オペレーティングシステムの機能を使ってみよう

19 / 24

fork-exec 方式 (16)

wait システムコール：親プロセスが子プロセスの終了を待つ。

書式 wait システムコールの書式を示す。

```
#include <sys/wait.h>
pid_t wait(int *status);
WIFEXITED(status) // 子プロセスがexitした(マクロ)
WIFSIGNALED(status) // 子プロセスがシグナルで終了した(マクロ)
WEXITSTATUS(status) // 子プロセスの終了ステータス(マクロ)
WTERMSIG(status) // 子プロセスを終了させたシグナルの番号(マクロ)
```

解説

- 終了したプロセスの番号 (PID: 正の値) を返す。
- エラーが発生した場合に-1を返す。
- status にプロセスの終了理由等が格納される。
status の内容は上記のマクロで読み取る。

オペレーティングシステムの機能を使ってみよう

20 / 24

fork-exec 方式 (17)

プログラム例

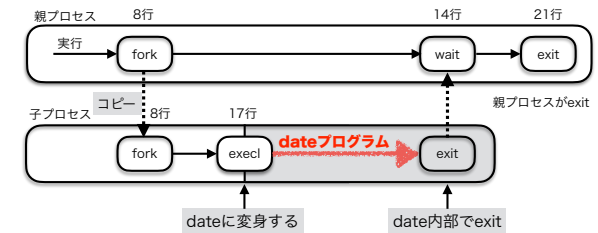
```
1 #include <stdio.h> // perror のために必要
2 #include <stdlib.h> // exit のために必要
3 #include <unistd.h> // fork, execve のために必要
4 #include <sys/wait.h> // wait のために必要
5 char *execpath="/bin/date";
6 int main(int argc, char *argv[], char *envp[]) {
7     int pid;
8     if ((pid=fork())<0) { // 分身を作る
9         perror(argv[0]); // fork がエラーなら
10        exit(1); // 親プロセスをエラー終了
11    }
12    if (pid!=0) { // pid が 0 以外なら自分は親プロセス
13        int status;
14        while (wait(&status)!=pid) // 子プロセスが終了するのを待つ
15            ;
16    } else { // pid が 0 なら自分は子プロセス
17        execl(execpath, "date", NULL); // date プログラムを実行 (execl を使用して)
18        perror(execpath); // exec が戻ってくるならエラー
19        exit(1); // エラー時はここで子プロセスを終了
20    }
21    // 親プロセスを正常終了
22    exit(0);
23 }
```

オペレーティングシステムの機能を使ってみよう

21 / 24

fork-exec 方式 (18)

プログラムの解説



オペレーティングシステムの機能を使ってみよう

22 / 24

fork-exec 方式 (19)

```
5 char *execpath="/bin/date";
6 int main(int argc, char *argv[], char *envp[]) {
7     int pid;
8     for (int i=1; argv[i]!=NULL; i++) {
9         if ((pid=fork())<0) { // 分身を作る
10            perror(argv[0]); // fork がエラーなら
11            exit(1); // 親プロセスをエラー終了
12        }
13        if (pid!=0) { // pid が 0 以外なら自分は親プロセス
14            int status;
15            while (wait(&status)!=pid) // 子プロセスが終了するのを待つ
16                ;
17        } else { // pid が 0 なら自分は子プロセス
18            putenv(argv[i]); // 環境変数を変更する
19            execl(execpath, "date", NULL); // date プログラムをロード・実行
20            perror(execpath); // execl が戻ってくるならエラー
21            exit(1); // エラー時はここで子プロセスを終了
22        }
23    }
24    // 親プロセスを正常終了
25    exit(0);
26 }
```

```
27 /* 実行例
28 % ./forkexec2 LC_TIME=ja_JP.UTF-8 LC_TIME=ru_RU.UTF-8 TZ=Cuba
29 2023年 4月 2日 日曜日 10時47分31秒 JST
30 в о с к р е с е н ь е , 2 а п р е л я 2023 г. 10:47:31 (JST)
31 Sat Apr 1 21:47:31 CDT 2023
32 */
```

オペレーティングシステムの機能を使ってみよう

23 / 24

fork-exec 方式 (20)

```
5 char *execpath="/bin/date";
6 int main(int argc, char *argv[], char *envp[]) {
7     int pid;
8     for (int i=1; argv[i]!=NULL; i++) {
9         putenv(argv[i]); // 環境変数を変更する
10        if ((pid=fork())<0) { // 分身を作る
11            perror(argv[0]); // fork がエラーなら
12            exit(1); // 親プロセスをエラー終了
13        }
14        if (pid!=0) { // pid が 0 以外なら自分は親プロセス
15            int status;
16            while (wait(&status)!=pid) // 子プロセスが終了するのを待つ
17                ;
18        } else { // pid が 0 なら自分は子プロセス
19            execl(execpath, "date", NULL); // date プログラムを実行 (execl を使用して)
20            perror(execpath); // exec が戻ってくるならエラー
21            exit(1); // エラー時はここで子プロセスを終了
22        }
23    }
24    // 親プロセスを正常終了
25    exit(0);
26 }
```

```
27 /* 実行例
28 % ./forkexec3 LC_TIME=ja_JP.UTF-8 LC_TIME=ru_RU.UTF-8 TZ=Cuba
29 2023年 4月 2日 日曜日 10時49分17秒 JST
30 в о с к р е с е н ь е , 2 а п р е л я 2023 г. 10:49:17 (JST)
31 с у б б о т а , 1 а п р е л я 2023 г. 21:49:17 (CDT)
32 */
```

オペレーティングシステムの機能を使ってみよう

24 / 24

オペレーティングシステムの機能を使ってみよう 第10章 UNIX シェル

オペレーティングシステムの機能を使ってみよう

1 / 7

UNIX のシェルとは

- CLI (Command Line Interface) 方式のコマンドインタプリタ
- macOS の Finder や Windows の Explorer は GUI 方式のシェル
- CLI 版のシェルは, sh, bash, ksh, zsh, csh, tcsh など
- UNIX シェルの持つべき機能
 1. 外部コマンド (プログラム) の起動
 2. カレントディレクトリの変更
 3. 環境変数の管理
 4. 入出力のリダイレクト, パイプ (<, >, |)
 5. ジョブの管理 (jobs, fg, bg など)
 6. ファイル名の展開 (ワイルドカード (*, ?))
 7. 繰り返しや条件判断
 8. スクリプトの実行 (処理の自動化)

オペレーティングシステムの機能を使ってみよう

2 / 7

簡易 UNIX シェル (myshell)

- 特徴
C 言語で 70 行以内で記述できる簡易シェル
- できること
 1. 外部コマンド (プログラム) の起動
 2. カレントディレクトリの変更
- 目的
myshell の構造を学び, 「fork-exec 方式」, 「環境変数」, 「リダイレクト」等への理解を深める.
- 目標
「環境変数の管理機能」, 「リダイレクト機能」を myshell に追加できるようにする.

オペレーティングシステムの機能を使ってみよう

3 / 7

基本構造 (main() 関数)

```

1 int main() {
2     char buf[MAXLINE+2];           // コマンド行を格納する配列
3     char *args[MAXARGS+1];         // 解析結果を格納する文字列配列
4     for (;;) {
5         printf("Command: ");        // プロンプトを表示する
6         if (fgets(buf, MAXLINE+2, stdin) == NULL) { // コマンド行を入力する
7             printf("\n");           // EOF なら
8             break;                  // 正常終了する
9         }
10        if (strchr(buf, '\n') == NULL) { // '\n' がバッファにない場合は
11            fprintf(stderr, "行が長すぎる\n"); // コマンド行が長すぎたので
12            return 1;                 // 異常終了する
13        }
14        if (!parse(buf, args)) {      // コマンド行を解析する
15            fprintf(stderr, "引数が多すぎる\n"); // 文字列が多すぎる場合は
16            continue;                 // ループの先頭に戻る
17        }
18        if (args[0] != NULL) execute(args); // コマンドを実行する
19    }
20    return 0;
21 }

```

オペレーティングシステムの機能を使ってみよう

4 / 7

コマンド行の解析 (parse() 関数) (1)

```

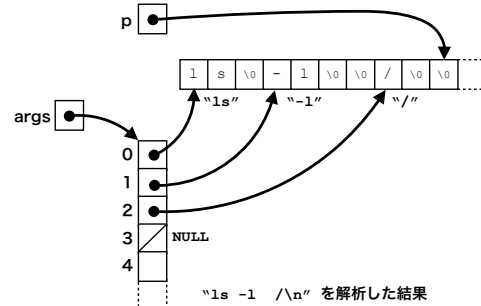
1 int parse(char *p, char *args[]) { // コマンド行を解析する
2     int i=0;                       // 解析後文字列の数
3     for (;;) {
4         while (isspace(*p)) *p++ = '\0'; // 空白を'\0'に書換える
5         if (*p == '\0' || i >= MAXARGS) break; // コマンド行の終端に到達で終了
6         args[i++] = p;              // 文字列を文字列配列に記録
7         while (*p != '\0' && !isspace(*p)) p++; // 文字列の最後まで進む
8     }
9     args[i] = NULL;                // 文字列配列の終端マーク
10    return *p == '\0';              // 解析完了なら 1 を返す
11 }

```

オペレーティングシステムの機能を使ってみよう

5 / 7

コマンド行の解析 (parse() 関数) (2)



オペレーティングシステムの機能を使ってみよう

6 / 7

コマンドの実行 (execute() 関数)

```
1 void execute(char *args[]) {           // コマンドを実行する
2     if (strcmp(args[0], "cd")==0) {     // cd (内部コマンド)
3         if (args[1]==NULL || args[2]!=NULL) { // 引数を確認して
4             fprintf(stderr, "Usage: cd DIR\n"); // 過不足ありなら使い方表示
5         } else if (chdir(args[1])<0) {    // 親プロセスが chdir する
6             perror(args[1]);             // chdirに失敗したらperror
7         }
8     } else {                             // 外部コマンドなら
9         int pid, status;
10        if ((pid = fork()) < 0) {          // 新しいプロセスを作る
11            perror("fork");
12            exit(1);
13        }
14        if (pid==0) {                     // 子プロセスなら
15            execvp(args[0], args);        // コマンドを実行
16            perror(args[0]);
17            exit(1);
18        } else {                          // 親プロセスなら
19            while (wait(&status) != pid)   // 子の終了を待つ
20                ;
21        }
22    }
23 }
```